UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Branch: MCA (Data Science)	**Semester:** 2nd
Student Name: Drishti Bhatia	**UID:** 25MCD10008
Subject Name: Technical Training - I Lab	**Subject Code**: 25CAP-652
Section/Group: 25MCD- KAR1	**Date of Performance:** 27-02-2026

Experiment No.: 6

1. **Aim of the practical:**
   Learn how to create, query, and manage views in SQL to simplify database queries and provide a layer of abstraction for end-users. **(Company Tags: Amazon, Zoho, ServiceNow)**

2. **Tools Used:** PostgreSQL

3. **Objectives of the practical:**
   - **Data Abstraction:** To understand how to hide complex table joins and calculations behind a simple virtual table interface.
   - **Enhanced Security:** To learn how to restrict user access to sensitive columns by providing views instead of direct table access.
   - **Query Simplification:** To master the creation of views that pre-join multiple tables, making reporting easier for non-technical users.
   - **View Management:** To understand the syntax for creating, altering, and dropping views, as well as the naming conventions required for efficient data access.

4. **Theory:**
   A **View** is essentially a virtual table based on the result-set of an SQL statement. It does not contain data of its own but dynamically pulls data from the underlying "base tables".
   1. **Simple Views:** Created from a single table without any aggregate functions or grouping. These are often updatable.
   2. **Complex Views:** Created from multiple tables using JOINs, or including GROUP BY and aggregate functions. These provide a consolidated summary of the database.
   3. **Security Layer:** In enterprise environments, views are used to grant permissions on specific subsets of data. For example, a "SalaryView" might exclude the "Employee_SSN" or "Home_Address" columns for privacy.
   4. **Benefits:** They simplify the user experience, ensure data consistency across reports, and reduce the risk of accidental data modification by providing read-only abstractions.

5. **Experiment / Practical Steps:**

Step 1: Creating a Simple View for Data Filtering

Implementing a view to provide a quick list of active employees without exposing the entire table structure.

Query:

```
--create tables
CREATE TABLE employees (
   emp_id SERIAL PRIMARY KEY,
   emp_name VARCHAR(100),
   salary NUMERIC(10,2),
   status VARCHAR(20),      -- Active / Inactive
   dept_id INT
);

CREATE TABLE departments (
   dept_id SERIAL PRIMARY KEY,
   dept_name VARCHAR(100)
);

-- Insert Departments
INSERT INTO departments (dept_name)
VALUES
('HR'),
('IT'),
('Finance'),
('Marketing');

-- Insert Employees
INSERT INTO employees (emp_name, salary, status, dept_id)
VALUES
('Amit Sharma', 50000, 'Active', 2),
('Neha Verma', 60000, 'Active', 3),
('Rahul Singh', 45000, 'Inactive', 1),
('Priya Mehta', 70000, 'Active', 2),
('Karan Malhotra', 55000, 'Inactive', 4);

--step1
CREATE VIEW active_employees AS
SELECT emp_id, emp_name, salary, dept_id
FROM employees
WHERE status = 'Active';

--view table
SELECT * FROM active_employees;
```

Output:

	emp_id integer	emp_name character varying (100)	salary numeric (10,2)	dept_id integer
1	1	Amit Sharma	50000.00	2
2	2	Neha Verma	60000.00	3
3	4	Priya Mehta	70000.00	2

Step 2: Creating a View for Joining Multiple Tables
Simplifying the retrieval of data distributed across Employees and Departments tables

Query:

```
--step2
CREATE VIEW employee_department_view AS
SELECT
   e.emp_id,
   e.emp_name,
   e.salary,
   e.status,
   d.dept_name
FROM employees e
JOIN departments d
ON e.dept_id = d.dept_id;


--view tables
SELECT * FROM employee_department_view;
```

Output:

	emp_id integer	emp_name character varying (100)	salary numeric (10,2)	status character varying (20)	dept_name character varying (100)
1	1	Amit Sharma	50000.00	Active	IT
2	2	Neha Verma	60000.00	Active	Finance
3	3	Rahul Singh	45000.00	Inactive	HR
4	4	Priya Mehta	70000.00	Active	IT
5	5	Karan Malhotra	55000.00	Inactive	Marketing

Step 3: Advanced Summarization View

Creating a view to provide department-level statistics automatically

Query:

```
--step3
CREATE VIEW department_summary AS
SELECT
   d.dept_name,
   COUNT(e.emp_id) AS total_employees,
   COUNT(CASE WHEN e.status = 'Active' THEN 1 END) AS active_employees,
   AVG(e.salary) AS avg_salary,
   SUM(e.salary) AS total_salary_expense
FROM departments d
LEFT JOIN employees e
ON d.dept_id = e.dept_id
GROUP BY d.dept_name;

--view tables
SELECT * FROM department_summary;
```

Output:

	dept_name character varying (100)	total_employees bigint	active_employees bigint	avg_salary numeric	total_salary_expense numeric
1	Marketing	1	0	55000.000000000000	55000.00
2	Finance	1	1	60000.000000000000	60000.00
3	IT	2	2	60000.000000000000	120000.00
4	HR	1	0	45000.000000000000	45000.00

6. Learning Outcomes:

- **Abstraction Proficiency:** Students will be able to create and query views to simplify efficient data access and abstraction.
- **Security Implementation:** Students will understand how to use views for data masking and providing restricted access to sensitive information.
- **Syntactic Accuracy:** Students will demonstrate the correct syntax for creating and querying views, ensuring logical clarity in naming conventions.
- **Real-world Application:** Students will be able to design views for practical domains like Library Management Systems or Payroll Systems to demonstrate functionality.